

动态图执行前向代码详解

目录

- 1 PythonAPI+pybind映射
- 2 Python C API 交互代码解析
- 3 dygraph function代码解析
- 4 C++ API
- 5 matmul反向节点代码

下面以matmul的python_c代码为示例，分析实现各APIpython_c接口需要的函数及功能

1 PythonAPI+pybind映射

从python API到c++代码的衔接原理在 [Python C API and Pybind交互](#) 文档中叙述。

2 Python C API 交互代码解析

PyObject：在Python语义中，每当我们实例化任意一个Python对象，在其占用的堆内存区块的首个字节就包含一个PyObject定义的副本，除PyObject的相关内存字节副本外，跟随PyObject对象内存副本之后的内存字节是当前对象的内存信息。

</> PyObject Python | 收起 ^

```
1 typedef struct_object{
2     _PyObject_HEAD_EXTRA
3     Py_ssize_t ob_refcnt;//引用计数器，和内存回收有关
4     PyTypeObject*ob_type;//定义Python对象的元类信息
5 }PyObject;
```

动态图的python_c接口代码：

整体功能为：获取tensor -> 获取Attributes -> 释放GIL锁 -> 获取执行硬件 -> 调用dygraph function -> 获取GIL锁-> 返回执行结果

</> python_c xxx_api C++ | 收起 ^

```
1 static PyObject * eager_api_matmul(PyObject *self, PyObject *args,
    PyObject *kwargs) {
```

```

2  paddle::platform::RecordEvent pythonc_record_event("matmul
pybind_imperative_func", paddle::platform::TracerEventType::UserDefined,
1);
3
4  PyThreadState *tstate = nullptr;
5  try {
6      VLOG(6) << "Running Eager Final State API: matmul";
7
8      // 【【【获取Tensors】】】
9      auto x = GetTensorFromArgs("matmul", "x", args, 0, false);
10     auto y = GetTensorFromArgs("matmul", "y", args, 1, false);
11
12     // 【【【获取Attributes】】】
13     PyObject* transpose_x_obj = PyTuple_GET_ITEM(args, 2);
14     bool transpose_x = CastPyArg2Boolean(transpose_x_obj, "matmul", 2);
15     PyObject* transpose_y_obj = PyTuple_GET_ITEM(args, 3);
16     bool transpose_y = CastPyArg2Boolean(transpose_y_obj, "matmul", 3);
17     //释放GIL锁
18     tstate = PyEval_SaveThread();
19
20     // 【【【进入c++执行】】】
21     // 设置硬件执行ID
22     auto place = egr::Controller::Instance().GetExpectedPlace();
23     if (paddle::platform::is_gpu_place(place)) {
24 #if defined(PADDLE_WITH_CUDA) || defined(PADDLE_WITH_HIP)
25         phi::backends::gpu::SetDeviceId(place.device);
26         VLOG(1) <<"CurrentDeviceId: " <<
phi::backends::gpu::GetCurrentDeviceId() << " from " <<
(int)place.device;
27 #else
28         PADDLE_THROW(paddle::platform::errors::PreconditionNotMet(
29             "PaddlePaddle should compile with GPU if use CUDAPlace."));
30 #endif
31     }
32     if (paddle::platform::is_custom_place(place)) {
33 #if defined(PADDLE_WITH_CUSTOM_DEVICE)
34         phi::DeviceManager::SetDevice(place);
35         VLOG(1) <<"CurrentDeviceId: " <<
phi::DeviceManager::GetDevice(place.GetDeviceType()) << " from " <<
(int)place.device;
36 #else
37         PADDLE_THROW(paddle::platform::errors::PreconditionNotMet(
38             "PaddlePaddle should compile with CUSTOM_DEVICE if use
CustomPlace."));

```

```

39 #endif
40 }
41
42 // 【调用dygraph function执行】
43 decltype(::matmul_dygraph_function(x,y,transpose_x,transpose_y)) out
44 = ::matmul_dygraph_function(x,y,transpose_x,transpose_y);
45
46 //获取GIL锁
47 PyEval_RestoreThread(tstate);
48 tstate = nullptr;
49 return ToPyObject(out);
50 } catch(...) {
51     if (tstate) {
52         PyEval_RestoreThread(tstate);
53     }
54     ThrowExceptionToPython(std::current_exception());
55     return nullptr;
56 }

```

1. 从arg中获得tensor的函数代码解析：

</> GetTensorFromArgs ()

C++ | 收起 ^

```

1 paddle::experimental::Tensor& GetTensorFromArgs(const std::string&
  op_type,
2
3                                     const std::string&
  arg_name,
4
5                                     PyObject* args,
6                                     ssize_t arg_idx,
7                                     bool dispensable) {
8     PyObject* obj = PyTuple_GET_ITEM(args, arg_idx);
9     return GetTensorFromPyObject(op_type, arg_name, obj, arg_idx,
10 dispensable);
11 }
12
13 //获取参数列表中的python实例
14 #define
15 PyTuple_GET_ITEM(op, i) (((PyTupleObject*)(op))->ob_item[i])
16
17 //做合理的非空检查
18 static paddle::experimental::Tensor& GetTensorFromPyObject(
19     const std::string& op_type,
20     const std::string& arg_name,
21     PyObject* obj,

```

```
19     ssize_t arg_idx,
20     bool dispensable) {
21     if (PyTuple_Check(obj)) {
22         obj = PyTuple_GET_ITEM(obj, 0);
23     }
24     //如果不是 (dispensable) 可有可无的tensor变量, 但是没有获取到实例报错
25     if (obj == nullptr || obj == Py_None) {
26         if (!dispensable) {
27             PADDLE_THROW(platform::errors::InvalidArgument(
28                 "%s(): argument '%s' (position %d) must be Tensor, but got
29                 None",
30                 op_type,
31                 arg_name,
32                 arg_idx));
33         }
34         static paddle::experimental::Tensor emptytensor;
35         return emptytensor;
36     }
37     // reinterpret_cast<类型说明符>(表达式)
38     //判断obj 是p_tensor_type 或 p_string_tensor_type 则将其转换为tensorObject
39     //并获取tensor实例
40     if (PyObject_IsInstance(obj, reinterpret_cast<PyObject*>
41         (p_tensor_type))) {
42         return reinterpret_cast<TensorObject*>(obj)->tensor;
43     } else if (PyObject_IsInstance(
44         obj, reinterpret_cast<PyObject*>
45         (p_string_tensor_type))) {
46         return reinterpret_cast<TensorObject*>(obj)->tensor;
47     } else {
48         PADDLE_THROW(platform::errors::InvalidArgument(
49             "%s(): argument '%s' (position %d) must be Tensor, but got %s",
50             op_type,
51             arg_name,
52             arg_idx,
53             reinterpret_cast<PyTypeObject*>(obj->ob_type)->tp_name));
54     }
55 }
```

2. 获取attributes

将PyObject 转为c++ object

</> op_function_common.cc

C++ | 收起 ^

```
1 bool CastPyArg2Boolean(PyObject* obj,
2                         const std::string& op_type,
3                         ssize_t arg_pos) {
4     if (obj == Py_None) {
5         return false; // To be compatible with QA integration testing. Some
6                       // test case pass in None.
7     } else if (obj == Py_True) {
8         return true;
9     } else if (obj == Py_False) {
10        return false;
11    } else {
12        PADDLE_THROW(platform::errors::InvalidArgument(
13            "%s(): argument (position %d) must be "
14            "bool, but got %s",
15            op_type,
16            arg_pos + 1,
17            ((PyTypeObject*)obj->ob_type)->tp_name)); // NOLINT
18    }
19
20    return false;
21 }
```

其他转换函数：

单个数据的转换(6类：bool ,int, int64, float, double, string)

</>

C++ | 收起 ^

```
1 bool CastPyArg2Boolean(PyObject* obj, const std::string& op_type,
2                         ssize_t arg_pos);
3 int CastPyArg2Int(PyObject* obj, const std::string& op_type, ssize_t
4   arg_pos);
5 int64_t CastPyArg2Long(PyObject* obj, const std::string& op_type,
6   ssize_t arg_pos);
7 float CastPyArg2Float(PyObject* obj, const std::string& op_type, ssize_t
8   arg_pos);
9 double CastPyArg2Double(PyObject* obj, std::string& op_type, ssize_t
10  arg_pos);
11 std::string CastPyArg2String(PyObject* obj, const std::string& op_type,
12  ssize_t arg_pos);
```

单个数据Map形式的转换

</>

C++ | 收起 ^

```
1 void CastPyArg2AttrXXX(PyObject* obj,
```

```
2         paddle::framework::AttributeMap& attrs,  
3         const std::string& key,  
4         const std::string& op_type,  
5         ssize_t arg_pos) {  
6     attrs[key] = CastPyArg2XXX(obj, op_type, arg_pos);  
7 }
```

vector数据的转换(List, Tuple, Sequence)

</>

C++ | 收起 ^

```
1 std::vector<XXX> CastPyArg2XXXs(PyObject* obj, const std::string&  
   op_type, ssize_t arg_pos){  
2     std::vector<XXX> value;  
3     if (PyList_Check(obj)) {  
4         Py_ssize_t len = PyList_Size(obj);  
5         PyObject* item = nullptr;  
6         for (Py_ssize_t i = 0; i < len; i++) {  
7             item = PyList_GetItem(obj, i);  
8             if (PyObject_CheckXXX(&item)) {  
9                 value.emplace_back(PyLong_AsLong(item));  
10            } else {  
11                PADDLE_THROW(platform::errors::InvalidArgument(  
12                    "%s(): argument (position %d) must be "  
13                    "list of bool, but got %s at pos %d",  
14                    op_type,  
15                    arg_pos + 1,  
16                    ((PyTypeObject*)item->ob_type)->tp_name, // NOLINT  
17                    i));  
18            }  
19        }  
20    } else if (PyTuple_Check(obj)) {  
21        Py_ssize_t len = PyTuple_Size(obj);  
22        PyObject* item = nullptr;  
23        for (Py_ssize_t i = 0; i < len; i++) {  
24            item = PyTuple_GetItem(obj, i);  
25            if (PyObject_CheckXXX(&item)) {  
26                value.emplace_back(PyLong_AsLong(item));  
27            } else {  
28                PADDLE_THROW(platform::errors::InvalidArgument(  
29                    "%s(): argument (position %d) must be "  
30                    "list of bool, but got %s at pos %d",  
31                    op_type,  
32                    arg_pos + 1,
```

```
33         ((PyTypeObject*)item->ob_type)->tp_name, // NOLINT
34         i));
35     }
36 }
37 } else if (PySequence_Check(obj)) {
38     Py_ssize_t len = PySequence_Size(obj);
39     PyObject* item = nullptr;
40     for (Py_ssize_t i = 0; i < len; i++) {
41         item = PySequence_GetItem(obj, i);
42         if (PyObject_CheckXXX(&item)) {
43             value.emplace_back(PyLong_AsLong(item));
44         } else {
45             PADDLE_THROW(platform::errors::InvalidArgument(
46                 "%s(): argument (position %d) must be "
47                 "list of int, but got %s at pos %d",
48                 op_type,
49                 arg_pos + 1,
50                 ((PyTypeObject*)item->ob_type)->tp_name, // NOLINT
51                 i));
52         }
53     }
54 } else if ((PyObject*)obj != Py_None) { // NOLINT
55     PADDLE_THROW(platform::errors::InvalidArgument(
56         "%s(): argument (position %d) must be "
57         "list or tuple, but got %s",
58         op_type,
59         arg_pos + 1,
60         ((PyTypeObject*)obj->ob_type)->tp_name)); // NOLINT
61 }
62
63 return value;
64 }
```

vector数据Map形式的转换

</>

C++ | 收起 ^

```
1 void CastPyArg2AttrXXXs(PyObject* obj,
2                          paddle::framework::AttributeMap& attrs,
3                          const std::string& key,
4                          const std::string& op_type,
5                          ssize_t arg_pos) {
6     attrs[key] = CastPyArg2XXXs(obj, op_type, arg_pos);
7 }
```

特殊转换：Block;VarBase

</>

C++ | 收起 ^

```
1 void CastPyArg2AttrBlock(PyObject* obj,
2                          paddle::framework::AttributeMap& attrs, //
3                          NOLINT
4                          const std::string& key,
5                          const std::string& op_type,
6                          ssize_t arg_pos)
```

</>

C++ | 收起 ^

```
1 std::shared_ptr<imperative::VarBase> GetVarBaseFromArgs(
2     const std::string& op_type,
3     const std::string& arg_name,
4     PyObject* args,
5     ssize_t arg_idx,
6     bool dispensable)
7
8 std::vector<std::shared_ptr<imperative::VarBase>>
9 GetVarBaseListFromArgs(
10     const std::string& op_type,
11     const std::string& arg_name,
12     PyObject* args,
13     ssize_t arg_idx,
14     bool dispensable)
```

创建参数Map

</>

C++ | 收起 ^

```
1 void ConstructAttrMapFromPyArgs(
2     const std::string& op_type,
3     PyObject* args,
4     ssize_t attr_start,
```



```

5         ssize_t attr_end,
6         paddle::framework::AttributeMap& attrs)

```

3. python中与线程GIL（全局解释锁）相关知识：

PyEval_SaveThread() 释放锁；PyEval_RestoreThread () 获取锁

python端的锁，python端无法同时并行，但当某一线程进入c++端时可以释放锁，使得其他线程的Python端继续运行。

4. 获取执行硬件

首先使用`egr::Controller::Instance().GetExpectedPlace()` 获取程序要的place,根据place的类型判断是否具有对应的编译环境 `paddle::platform::is_gpu_place(place)`;

`paddle::platform::is_custom_place(place)`

判断是否有GPU环境（CUDA,HIP）：若有使用 `phi::backends::gpu::SetDeviceId(place.device)`设置device ID

判断是否是custom_place（新增硬件）：若有使用`phi::DeviceManager::SetDevice(place)`设置device ID

</> 获取硬件代码

C++ | 收起 ^

```

1 // Set Device ID
2     auto place = egr::Controller::Instance().GetExpectedPlace();
3     if (paddle::platform::is_gpu_place(place)) {
4 #if defined(PADDLE_WITH_CUDA) || defined(PADDLE_WITH_HIP)
5         phi::backends::gpu::SetDeviceId(place.device);
6         VLOG(1) <<"CurrentDeviceId: " <<
phi::backends::gpu::GetCurrentDeviceId() << " from " <<
(int)place.device;
7 #else
8         PADDLE_THROW(paddle::platform::errors::PreconditionNotMet(
9             "PaddlePaddle should compile with GPU if use CUDAPlace."));
10 #endif
11     }
12     if (paddle::platform::is_custom_place(place)) {
13 #if defined(PADDLE_WITH_CUSTOM_DEVICE)
14         phi::DeviceManager::SetDevice(place);
15         VLOG(1) <<"CurrentDeviceId: " <<
phi::DeviceManager::GetDevice(place.GetDeviceType()) << " from " <<
(int)place.device;
16 #else
17         PADDLE_THROW(paddle::platform::errors::PreconditionNotMet(
18             "PaddlePaddle should compile with CUSTOM_DEVICE if use
CustomPlace."));

```

```
19 #endif
20 }
```

5. 调用dygraph function

`decltype(::matmul_dygraph_function(x,y,transpose_x,transpose_y))` 是修饰返回tensor out 的参数类型 Tensor （其中的kernel调用不会执行吗）

`decltype` : `class paddle::experimental::Tensor`

dygraph function 存放在 `paddle/fluid/api/generated/forward/dygraph_functions.cc`

```
</> python_c xxx_api C++ 收起 ^
1 decltype(::matmul_dygraph_function(x,y,transpose_x,transpose_y)) out =
  ::matmul_dygraph_function(x,y,transpose_x,transpose_y);
```

6. 获取GIL锁，在python端执行。

3 dygraph function代码解析

AMP逻辑 -> 获取输入的AutoGradMeta信息 -> 调用C++ API 获取输出 -> 创建输出的AutoGradMeta信息

```
</> C++ 收起 ^
1 paddle::experimental::Tensor matmul_dygraph_function(const
  paddle::experimental::Tensor& x, const paddle::experimental::Tensor& y,
  bool transpose_x, bool transpose_y) {
2   // Dygraph Record Event
3   paddle::platform::RecordEvent dygraph_entrance_record_event("matmul
  dygraph", paddle::platform::TracerEventType::Operator, 1);
4
5   // 【AMP Logic】
6   if (egr::Controller::Instance().GetAMPLevel() !=
  paddle::imperative::AmpLevel::00) {
7     VLOG(5) << "Check and Prepare For AMP";
8     auto op_name = phi::TransToFluidOpName("matmul");
9     paddle::small_vector<std::vector<paddle::experimental::Tensor>,
  egr::kSlotSmallVectorSize> amp_tensors_vector = { {x},{y} };
10
11     auto amp_dst_dtype = egr::GetAmpDestDtype(op_name,
  amp_tensors_vector);
12
13     auto NEW_x = egr::EagerAmpAutoCast("x", x, amp_dst_dtype, op_name);
14     auto NEW_y = egr::EagerAmpAutoCast("y", y, amp_dst_dtype, op_name);
15
16     {
```

```
17     paddle::imperative::AutoCastGuard
    guard(egr::Controller::Instance().GetCurrentTracer(),
    paddle::imperative::AmpLevel::00);
18     return matmul_dygraph_function(NEW_x, NEW_y, transpose_x,
    transpose_y);
19 }
20 }
21
22 // 【【【获取输入的AutoGradMeta信息】】】
23     egr::AutogradMeta* x_autograd_meta =
    egr::EagerUtils::nullable_autograd_meta(x);
24     egr::AutogradMeta* y_autograd_meta =
    egr::EagerUtils::nullable_autograd_meta(y);
25     // 【【【调用C++ API】】】
26     VLOG(3) << "Final State Running: " << "matmul_dygraph_function";
27     auto api_result = paddle::experimental::matmul(x, y, transpose_x,
    transpose_y);
28     // Check NaN and Inf if needed
29     if (FLAGS_check_nan_inf) { egr::CheckTensorHasNanOrInf("matmul",
    api_result); }
30
31     auto& out = api_result;
32
33     // 【【【创建输出的AutoGradMeta信息】】】
34     egr::AutogradMeta* out_autograd_meta =
    egr::EagerUtils::autograd_meta(&out);
35     // 【【【判断是否需要建立反向图】】】
36     bool trace_backward = egr::Controller::Instance().HasGrad();
37     bool require_any_grad =
    egr::EagerUtils::ComputeRequireGrad(trace_backward, x_autograd_meta, y_autograd_meta);
38
39     // Check Inplace if needed
40
41     // Node Creation
42     if(require_any_grad) {
43         paddle::platform::RecordEvent node_creation_record_event("matmul
    node_creation", paddle::platform::TracerEventType::OperatorInner, 1);
44
45         egr::EagerUtils::PassStopGradient(false, out_autograd_meta);
46
47         // Node Construction
48         auto grad_node = std::shared_ptr<MatmulGradNodeFinal>(new
    MatmulGradNodeFinal(1, 2));
```

```

49 // SetAttributes if needed
50 grad_node->SetAttributeTranspose_x(transpose_x);
51 grad_node->SetAttributeTranspose_y(transpose_y);
52 // Set TensorWrappers for Forward Inputs if needed
53 grad_node->SetTensorWrapperx(x);
54 grad_node->SetTensorWrappery(y);
55 // SetGradOutMeta & SetEdges
56 grad_node->SetGradOutMeta(x, 0);
57 grad_node->SetGradOutMeta(y, 1);
58 // SetOutRank & SetHistory & SetGradInMeta & RetainGrad
59 if (out_autograd_meta) {
60     egr::EagerUtils::SetOutRankWithSlot(out_autograd_meta, 0);
61 }
62 if (out_autograd_meta) {
63     egr::EagerUtils::SetHistory(out_autograd_meta, grad_node);
64 }
65 grad_node->SetGradInMeta(out, 0);
66 egr::EagerUtils::CheckAndRetainGrad(out);
67 // Set TensorWrappers for Forward Outputs if needed
68
69 }
70
71 // Returns
72 return out;
73 }
74

```

1. AMP 逻辑：

首先添加混合精度处理逻辑（AMP），在判断支持AMP的分支下实现AMP的准备工作，添加VLOG便于调试是否进入混精模式。

- 1>定义输入tensor的vector：amp_tensors_vector
- 2>使用`egr::GetAmpDestDtype` 函数从tensor中获得各个输入tensor的数据类型
amp_dst_dtype
- 3>使用`egr::EagerAmpAutoCast`函数分别对所有输入tensor进行数据类型转换（F32->F16）
获得新的输入Tensor。
- 4>在{}代码块中先定义AutoCastGuard,参数为分别为
`egr::Controller::Instance().GetCurrentTracer()`当前执行
`tracer,paddle::imperative::AmpLevel::O0`);设置精度模式为F32,以便后续调用本函数跳过
AMP准备模块;
调用本函数xxx_dygraph_function,参数为转换后的新输入tensor及attrmap;

注：AMP模式有4种，目前只有01，02有效

```
enum class AmpLevel {
```

```
    O0 = 0, // fp32
```

```
    O1,     // amp, mixed fp32-fp16
```

```
    O2,     // almost fp16
```

```
    O3,     // fp16
```

```
};
```

2. 获取输入的AutoGradMeta (反向节点输入信息)：

使用`egr::EagerUtils::nullable_autograd_meta()` 获取输入tensor的反向计算信息 (AutoGradMeta),参数是输入tensor。运行返回空指针

```
</>
```

```
C++
```

```
| 收起 ^
```

```
1 AutogradMeta*EagerUtils::nullable_autograd_meta(  
2     constpaddle::experimental::Tensor&target) {  
3     auto* p_autograd_meta = target.get_autograd_meta();  
4     if (!p_autograd_meta) return nullptr;  
5     returnstatic_cast<AutogradMeta*>(p_autograd_meta);  
6 }
```

其他相关函数

```
</>
```

```
C++
```

```
| 收起 ^
```

```
1 //如果没有获取到AutoGradMeta则创建一个实例，使用共享指针获取。  
2 AutogradMeta* EagerUtils::autograd_meta(paddle::experimental::Tensor*  
3     target)  
3 //不允许修改target,返回强制类型转换得到的AutoGradMeta指针  
4 AutogradMeta* EagerUtils::unsafe_autograd_meta(const  
5     paddle::experimental::Tensor& target)
```

3. 调用C++ API, 获取输出后检查输出是否合法

c++API 由 yami配置自动生成，声明存放在`paddle/phi/api/include/api.h`下 (反向`paddle/phi/api/backward/backward_api.h`)；

c++ API 的实现存放在`paddle/phi/api/lib/api.cc` (反向`paddle/phi/api/lib/backward_api.h`)

4. 创建输出的AutoGradMeta信息

调用`autograd_meta()` 为输出tensor创建存放反向计算信息的AutoGradMeta

5. 判断是否需要建立反向图

`egr::Controller::Instance().HasGrad()` 判断tracer是否需要梯度 (with `no_grad()` 设置某一段计算不需要反向)

`egr::EagerUtils::ComputeRequireGrad(trace_backward,x_autograd_meta,y_autograd_meta)`取

tracer,输入tensor是否需要梯度的并集。

6. 创建反向节点及反向图关系

(创建反向图的过程需要了解相关数据结构, 在文档 [动态图自动微分模块类的设计](#) 中详细介绍了相关数据结构以及为什么需要这些数据结构)

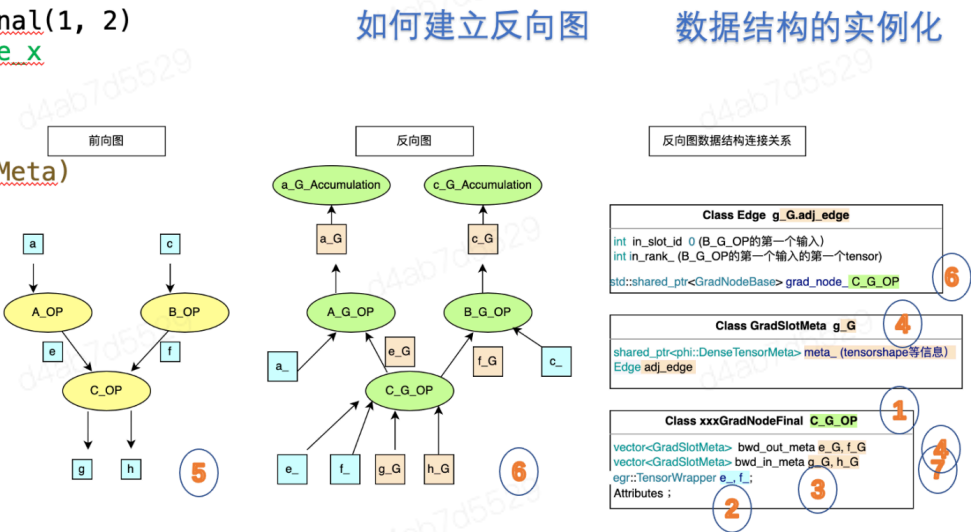
基于以上数据结构, 反向图的建立过程本质上是数据结构的实例化, 其中7个调用函数对应在数据结构上的构造关系如下图所示:

图中以c_op节点的反向图结构建立为例。在执行C_OP代码后首先1创建C_G_OP反向节点; 2,3设置其中attribute,tensorwrapper变量; 4设置输出的meta信息。

5对输出的tensor设置autogradMeta信息, 绑定反向op,6设置meta中edge的下一结点, 7设置输入的meta信息。具体的代码分析在后面详细介绍。

- 1.new MatmulGradNodeFinal(1, 2)
- 2.SetAttributetranspose_x
- 3.SetTensorWrapperx
- 4.SetGradOutMeta
- 5.SetOutRankWithSlot
(设置Tensor的AutoGradMeta)
- 6.SetHistory
- 7.SetGradInMeta

GradNodeBase基类函数
GradNodeBase子类函数
Eager.utils 函数



反向节点:

- 1> 调用PassStopGradient函数, 遍历输出tensor的AutoGradMeta设置stop_gradient参数为false
- 2> 创建grad_node, 先new一个反向函数类的实例, 参数为此反向函数的输入参数个数和输出参数个数: new XXXGradNodeCompat(1, 3), 再使用shared_ptr创建grad_node, 指向此反向函数的实例。
- 3> 调用grad_node的SetAttributetranspose_x() 函数设置attributes,
- 4> 调用grad_node的SetTensorWrapperX(), SetTensorWrapperY()函数设置反向函数中的TensorWrapper变量, (当前向输入的Tensor作为反向的输入时, 不需要记录自己的反向信息, 因此使用TensorWrapper,只对AutoGradMeta信息进行弱引用) 参数是对应的输入tensor。

```
</> C++ | 收起 ^

1 void SetTensorWrapperx(const paddle::experimental::Tensor& x) {
2     x_ = egr::TensorWrapper(x, false);
3 }
4
5 explicit TensorWrapper(const paddle::experimental::Tensor& tensor,
6                         bool no_need_buffer = false) {
```



```

7 // set inplace_version_snapshot_ according to tensor's current
inplace
8 // version.
9 if (tensor.impl() && phi::DenseTensor::classof(tensor.impl().get()))
{
10     phi::DenseTensor* dense_tensor =
11         static_cast<phi::DenseTensor*>(tensor.impl().get());
12     auto& inplace_version_counter = dense_tensor->
InplaceVersionCounter();
13     inplace_version_snapshot_ =
inplace_version_counter.CurrentVersion();
14 }
15
16 /**
17  * Normally, we should only save data and part of autograd_meta of
fwd
18  * tensor, and should not reserve its original grad_node,
19  * to avoid recursive and additional depends on GradNodeBase
20  * **/
21     auto* tensor_autograd_meta =
EagerUtils::nullable_autograd_meta(tensor);
22     no_need_buffer_ = no_need_buffer;
23     // shallow copy tensor_impl here
24     if (no_need_buffer) {
25         if (phi::DenseTensor::classof(tensor.impl().get())) {
26             // Only Copy Meta
27             phi::DenseTensor* dense_tensor =
28                 static_cast<phi::DenseTensor*>(tensor.impl().get());
29             // TODO(jiabin): It's not a good idea to set memory size to
zero, find
30             // another way and change this.
31             intermediate_tensor_.set_impl(
32                 std::move(std::make_shared<phi::DenseTensor>(
33                     std::make_shared<phi::Allocation>(nullptr, 0,
tensor.place()),
34                     std::move(dense_tensor->meta()))));
35         } else {
36             PADDLE_THROW(paddle::platform::errors::Fatal(
37                 "Unrecognized tensor type for no_need_buffer feature"));
38         }
39     } else { //复制tensor的数据部分
40 #ifndef PADDLE_NO_PYTHON
41         if (SavedTensorsHooks::GetInstance().IsEnable() &&
42             tensor.is_dense_tensor()) {

```

```

43     phi::DenseTensor* dense_tensor =
44         static_cast<phi::DenseTensor*>(tensor.impl().get());
45     intermediate_tensor_.set_impl(
46         std::move(std::make_shared<phi::DenseTensor>(
47             std::make_shared<phi::Allocation>(nullptr, 0,
48             tensor.place()),
49             dense_tensor->meta())));
50     auto pack_hook = SavedTensorsHooks::GetInstance().GetPackHook();
51     unpack_hook_ = SavedTensorsHooks::GetInstance().GetUnPackHook();
52     packed_value_ = reinterpret_cast<PyObject*>((*pack_hook)
53         (tensor));
54 } else {
55 #endif
56     intermediate_tensor_.set_impl(tensor.impl());
57 #ifndef PADDLE_NO_PYTHON
58     }
59 #endif
60     if (VLOG_IS_ON(7)) {
61         // TODO(jiabin): This may has server performance issue
62         intermediate_tensor_.set_name(tensor.name() + "@Saved");
63     }
64     //创建对AutoGradMeta的弱引用
65     if (tensor_autograd_meta) {
66         auto autograd_meta =
67             std::make_shared<AutogradMeta>(*tensor_autograd_meta);
68         autograd_meta->ResetGradNode();
69         intermediate_tensor_.set_autograd_meta(autograd_meta);
70         weak_grad_node_ = tensor_autograd_meta->GetMutableGradNode();
71     }
72 }

```

5> 调用grad_node的SetGradOutMeta () 函数分别设置反向各个输出tensor的信息，参数分别为前向输入tensor,和此输入tensor是前向算子的第几个输入。(前向的输入tensor的梯度将会成为反向的输出tensor，且顺序一致，此函数中会对前向输入tensor的autogradmeta信息进行设置)

</> SetGradOutMeta

C++ | 收起 ^

```

1 void GradNodeBase::SetGradOutMeta(const paddle::experimental::Tensor&
2     fwd_in,
3     size_t slot_rank) {
4     //获得前向输入的AutoGradMeta,输出的id slot_rank不能比反向节点中设置的输出个数多

```



```
4 auto* fwd_in_meta = egr::EagerUtils::nullable_autograd_meta(fwd_in);
5 PADDLE_ENFORCE_LE(
6     (slot_rank + 1),
7     bwd_out_meta_.size(),
8     paddle::platform::errors::InvalidArgument(
9         "Slot Rank should less equal than bwd_out_meta_ size, "
10        "since bwd_out_meta_ is designed to hold as same num as "
11        "backward outputs."));
12 //获得反向节点中输出meta 的第slot_rank个输出
13 auto& metas = bwd_out_meta_.at(slot_rank);
14 // Init stop gradient vector before use to avoid push back
15 if (metas.size() == 0) {
16     metas.resize(1);
17 }
18 auto& meta = metas[0];
19 // 设置stop_gradient信息
20 if (fwd_in_meta) {
21     meta.SetStopGradient(fwd_in_meta->StopGradient());
22 } else {
23     meta.SetStopGradient(true);
24 }
25 // 建立从本节点到下一个节点的Edges
26 if (fwd_in_meta && !fwd_in_meta->StopGradient()) {
27     auto node = fwd_in_meta->GetMutableGradNode();
28     //如果前向输入没有连接下一个反向节点，则设置一个Accumulation节点
29     if (!node || !node.get()) {
30         fwd_in_meta->SetGradNode(
31             std::make_shared<egr::GradNodeAccumulation>(fwd_in_meta));
32     }
33     VLOG(3) << "Add Edges for slot: " << slot_rank << ", the Edge is
from "
34         << this->name() << " (addr: " << this << ") "
35         << " to " << fwd_in_meta->GetMutableGradNode()->name()
36         << " (addr: " << fwd_in_meta->GetMutableGradNode().get() <<
");";
37     //在反向节点的bwd_in_meta中的meta信息中增加与上一节点连接的边
38     meta.SetEdge(fwd_in_meta->GetMutableGradNode(), fwd_in_meta-
>OutRankInfo());
39 }
40 // Record TensorMeta
41 if (fwd_in.impl() && fwd_in.impl().get()) {
42     if (phi::DenseTensor::classof(fwd_in.impl().get())) {
43         // Only Copy Meta
```

```

44     phi::DenseTensor* dense_tensor =
45         static_cast<phi::DenseTensor*>(fwd_in.impl().get());
46     PADDLE_ENFORCE_NE(
47         dense_tensor->meta().dtype,
48         phi::DataType::UNDEFINED,
49         paddle::platform::errors::Fatal("Attempting to copy
DenseTensorMeta "
50                                         "with
phi::DataType::UNDEFINED,"
51                                         "which is illegal."));
52     meta.SetTensorMeta(dense_tensor->meta());
53     meta.SetPlace(fwd_in.place());
54 }
55 } else {
56     VLOG(7) << "Unable to initialize the DenseTensorMeta of GradSlotMeta
with "
57               "non-DenseTensor argument.";
58 }
59 }

```

注：3，4中调用的函数在反向节点的类中实现，每个参数和输入,输出都有对应的Set函数, 5中调用的函数在反向节点的基类中实现。

反向图：

6> 调用`egr::EagerUtils::SetOutRankWithSlot`对前向输出tensor的autogradmeta信息进行标记，参数分别为第4步中创建的输出梯度`p_autograd_Out`，及此输出是前向第几个输出信息。

</> SetOutRankWithSlot

C++ | 收起 ^

```

1 void EagerUtils::SetOutRankWithSlot(AutogradMeta* target, size_t
slot_id) {
2     target->SetSingleOutRankWithSlot(slot_id, 0);
3 }
4 //设置输出tensor的 AutoGradMeta信息中的输出位置
5 void SetSingleOutRankWithSlot(size_t slot_id, size_t rank) {
6     out_slot_id_ = slot_id;
7     out_rank_ = rank;
8 }

```

7> 调用`egr::EagerUtils::SetHistory`绑定前向输出tensor的autogradmeta和此反向节点`grad_node`，参数分别为第4步中创建的输出梯度`p_autograd_Out`,及第6.3>步中创建的`grad_node`。

</>

C++ | 收起 ^

```
1 void EagerUtils::SetHistory(AutogradMeta* autograd_meta,
2                             const std::shared_ptr<GradNodeBase>&
3                             grad_node) {
4     设置输出tensor的AutogradMeta信息中的反向节点信息
5     if (autograd_meta->GradNode()) {
6         VLOG(7) << "Should not set grad node twice, original node is:"
7         << autograd_meta->GradNode()->name()
8         << "current is: " << grad_node->name();
9     }
10    autograd_meta->SetGradNode(grad_node);
11 }
```

8> 调用grad_node的SetGradInMeta () 函数分别设置反向各个输入tensor (梯度)的信息, 参数为前向输出tensor,和此输出tensor是前向算子的第几个输出。

</>

C++ | 收起 ^

```
1 void GradNodeBase::SetGradInMeta(const paddle::experimental::Tensor&
2     fwd_out,
3     size_t slot_rank) {
4     VLOG(7) << "Set GradSlotMeta for Grad Inputs";
5     auto* fwd_out_meta = egr::EagerUtils::nullable_autograd_meta(fwd_out);
6     PADDLE_ENFORCE_LE(
7         slot_rank,
8         (bwd_in_meta_.size() - 1),
9         paddle::platform::errors::InvalidArgument(
10             "Slot Rank should less equal than bwd_in_meta_ size, since "
11             "bwd_in_meta_ is designed to hold as same num as backward "
12             "inputs."));
13     auto& metas = bwd_in_meta_.at(slot_rank);
14     if (metas.size() == 0) {
15         metas.resize(1);
16     }
17     auto& meta = metas[0];
18     if (fwd_out_meta && fwd_out_meta->StopGradient()) {
19         meta.SetStopGradient(fwd_out_meta->StopGradient());
20     }
21     if (!fwd_out.initialized()) {
22         VLOG(7)
23         << "Skip Configuring GradSlotMeta for uninitialized GradInput
24         Tensor";
25     }
26     return;
```

```

26 }
27
28 phi::DenseTensor* dense_tensor = nullptr;
29 // Record TensorMeta
30 if (phi::DenseTensor::classof(fwd_out.impl().get())) {
31     // Only Copy Meta
32     dense_tensor = static_cast<phi::DenseTensor*>(fwd_out.impl().get());
33 } else if (phi::SparseCooTensor::classof(fwd_out.impl().get())) {
34     phi::SparseCooTensor* coo_tensor =
35         static_cast<phi::SparseCooTensor*>(fwd_out.impl().get());
36     dense_tensor = coo_tensor->mutable_non_zero_elements();
37 } else if (phi::SparseCsrTensor::classof(fwd_out.impl().get())) {
38     phi::SparseCsrTensor* csr_tensor =
39         static_cast<phi::SparseCsrTensor*>(fwd_out.impl().get());
40     dense_tensor = csr_tensor->mutable_non_zero_elements();
41 } else {
42     VLOG(7) << "Unable to initialize the DenseTensorMeta of GradSlotMeta
with "
43         "non-DenseTensor argument.";
44 }
45 PADDLE_ENFORCE_NE(
46     dense_tensor->meta().dtype,
47     phi::DataType::UNDEFINED,
48     paddle::platform::errors::Fatal(
49         "Attempting to copy DenseTensorMeta with
phi::DataType::UNDEFINED,"
50         "which is illegal."));
51
52 meta.SetTensorMeta(dense_tensor->meta());
53 meta.SetPlace(fwd_out.place());
54
55 if (dense_tensor->type() == paddle::experimental::DataType::COMPLEX64
||
56     dense_tensor->type() ==
paddle::experimental::DataType::COMPLEX128) {
57     need_complex_to_real_ = true;
58 }
59 }

```

9> 调用`egr::EagerUtils::CheckAndRetainGrad()` 设置是否保留前向输出tensor的梯度信息，参数是前向输出tensor。（当此tensor是叶子tensor时默认会保留，否则根据用户设置判断是否保留，同时设置hook。

```
1 void EagerUtils::CheckAndRetainGrad(
2     const std::vector<paddle::experimental::Tensor*>& tensors) {
3     if (FLAGS_retain_grad_for_all_tensor) {
4         for (auto& tensor : tensors) {
5             VLOG(6) << "RetainGradForTensor: " << tensor->name();
6             egr::egr_utils_api::RetainGradForTensor(*tensor);
7         }
8     }
9 }
10
11
12 void RetainGradForTensor(const paddle::experimental::Tensor& tensor) {
13     if (IsLeafTensor(tensor)) {
14         // Leaf tensor's grad will always be retained
15         // Refer to implementation of AccumulationNode for more details
16         return;
17     } else {
18         AutogradMeta* meta = EagerUtils::unsafe_autograd_meta(tensor);
19         if (meta->RetainGrads()) {
20             return;
21         } else {
22             meta->SetRetainGrads(true);
23         }
24
25         std::weak_ptr<paddle::experimental::Tensor> weak_grad_tensor =
26             meta->WeakGrad();
27
28         // Define Hook
29         auto hook = [weak_grad_tensor](const paddle::experimental::Tensor&
30             t) {
31             if (!weak_grad_tensor.expired()) {
32                 auto grad_tensor = weak_grad_tensor.lock();
33                 if (t.defined()) {
34                     VLOG(7) << "Set impl for RetainGrad Hook for tensor: " <<
35                         t.name();
36                     // Simply Copy impl() to grad_tensor
37                     grad_tensor->set_impl(t.impl());
38                     grad_tensor->set_autograd_meta(t.mutable_autograd_meta());
39                     return *grad_tensor.get();
40                 } else {
41                     VLOG(7) << "Retain NULL paddle::experimental::Tensor in Grad
42                         Hook";
43                     return paddle::experimental::Tensor();
44                 }
45             }
46         };
47     }
48 }
```

```
41     }
42     } else {
43         VLOG(7) << "Retain NULL paddle::experimental::Tensor in Grad
Hook";
44         return paddle::experimental::Tensor();
45     }
46 };
47
48 // Append to GradientHooks
49 RegisterGradientHookForTensor(tensor, hook);
50 }
51 }
```

4 C++ API

根据输入tensor获取kernel的信息 (backend,layout,datatype) -> 根据信息选择kernel, 获得kernel -> 获取执行硬件, 输入tensor数据 -> 调用InferMeta 函数 -> 获得kernel的声明格式 -> 调用运算kernel函数 -> 如果需要返回cpu端, 将输出转换为cpu数据

</>

C++ | 收起 ^

```
1 PADDLE_API Tensor matmul(const Tensor& x, const Tensor& y, bool
transpose_x, bool transpose_y) {
2
3     Backend kernel_backend = Backend::UNDEFINED;
4     DataLayout kernel_layout = DataLayout::UNDEFINED;
5     DataType kernel_data_type = DataType::UNDEFINED;
6     // 【【【根据输入tensor获取kernel的信息】】】
7     if (kernel_backend == Backend::UNDEFINED
8         || kernel_layout == DataLayout::UNDEFINED
9         || kernel_data_type == DataType::UNDEFINED ) {
10         auto kernel_key_set = ParseKernelKeyByInputArgs(x, y);
11         auto kernel_key = kernel_key_set.GetHighestPriorityKernelKey();
12         if (kernel_backend == Backend::UNDEFINED) {
13             kernel_backend = kernel_key.backend();
14         }
15         if (kernel_layout == DataLayout::UNDEFINED) {
16             kernel_layout = kernel_key.layout();
17         }
18         if (kernel_data_type == DataType::UNDEFINED) {
19             kernel_data_type = kernel_key.dtype();
20         }
21     }
```

```
22
23  VLOG(6) << "matmul API kernel key: [" << kernel_backend << ", " <<
kernel_layout << ", "<< kernel_data_type << "];";
24  // 【【【选择kernel, 获得kernel】】】
25  auto kernel_result =
phi::KernelFactory::Instance().SelectKernelOrThrowError(
26      "matmul", {kernel_backend, kernel_layout, kernel_data_type});
27  const auto& kernel = kernel_result.kernel;
28  VLOG(6) << "matmul kernel: " << kernel;
29
30  // 【【【获取执行硬件】】】
31  auto* dev_ctx =
GetDeviceContextByBackend(kernel_result.has_fallback_cpu ? Backend::CPU
: kernel_backend);
32  // 【【【获取输入tensor数据（如需转换place,layout,dtype会在此处处理）】】】
33  auto input_x = PrepareData(x, kernel.InputAt(0), {});
34  auto input_y = PrepareData(y, kernel.InputAt(1), {});
35  // 【【【 ? ? ? ? ? 】】】
36  if(platform::RecordOpInfoSupplement::IsEnabled()){
37      std::vector<std::pair<const char*, std::vector<phi::DDim>>>
input_shapes{
38          {"x", {
39              (*input_x).dims()}},
40          {"y", {
41              (*input_y).dims()}}};
42      platform::RecordOpInfoSupplement("matmul", input_shapes);
43  }
44  // 【【【 设置输出tensor的数据指针】】】
45  Tensor api_output;
46  auto kernel_out = SetKernelOutput(kernel_backend, &api_output);
47
48  paddle::platform::RecordEvent *infer_shape_record_event = nullptr;
49  if(paddle::platform::RecordEvent::IsEnabled()){
50      infer_shape_record_event = new paddle::platform::RecordEvent("matmul
infer_meta", paddle::platform::TracerEventType::OperatorInner, 1);
51  }
52  // 【【【调用InferMeta 函数判断输入是否合法, 为输出tensor设置dim,layout,dtype】】】
53  phi::MetaTensor meta_out(kernel_out);
54  phi::MatmulInferMeta(MakeMetaTensor(*input_x),
MakeMetaTensor(*input_y), transpose_x, transpose_y, &meta_out);
55
56  if(infer_shape_record_event != nullptr){
57      delete infer_shape_record_event;
58  }
```



```

59  // 【获得kernel的声明格式】
60  using kernel_signature = void (*)(const platform::DeviceContext&, const
    phi::DenseTensor&, const phi::DenseTensor&, bool, bool,
    phi::DenseTensor*);
61  auto* kernel_fn = kernel.GetVariadicKernelFn<kernel_signature>();
62
63  paddle::platform::RecordEvent* kernel_record_event = nullptr;
64  if(paddle::platform::RecordEvent::IsEnabled()){
65      kernel_record_event = new paddle::platform::RecordEvent("matmul
    compute", paddle::platform::TracerEventType::OperatorInner, 1);
66  }
67  // 【调用运算kernel函数】
68  (*kernel_fn)(*dev_ctx, *input_x, *input_y, transpose_x, transpose_y,
    kernel_out);
69  if(kernel_record_event != nullptr){
70      delete kernel_record_event;
71  }
72  // 【如果需要返回cpu端，将输出转换为cpu数据】
73  if (kernel_result.has_fallback_cpu) {
74
75      TransDataBackend(kernel_out, kernel_backend, kernel_out);
76  }
77  return api_output;
78 }

```

1. 根据输入tensor获取kernel的信息

</>

C++ | 收起 ^

```

1  auto kernel_key_set = ParseKernelKeyByInputArgs(x, y);
2  auto kernel_key = kernel_key_set.GetHighestPriorityKernelKey();
3
4  KernelKeySet ParseKernelKeyByInputArgs(const Args&... args) {
5      return detail::KernelKeyParser().apply(args...).key_set;
6  }
7
8  struct KernelKeyParser : ArgsIterator<KernelKeyParser> {
9      KernelKeySet key_set;
10     // this dtype_set is used for cache multi-inputs dtype and used for
11     // data_promote
12     DataTypeSet dtype_set{DataType::UNDEFINED};
13
14     // TODO(chenweihang): deal with multiple diff input Tensors
15     // TODO(chenweihang): add global device guard method to set backend
16     inline void AssignKernelKeySet(const phi::TensorBase& tensor) {

```

```
17     key_set.backend_set =
18         key_set.backend_set | detail::GetTensorBackendSet(tensor);
19     // TODO(chenweihang): select multi layout and dtype
20     key_set.layout = tensor.layout();
21     key_set.dtype = tensor.dtype();
22     dtype_set = dtype_set | DataTypeSet(key_set.dtype);
23     auto promote_result = PromoteTypes(dtype_set);
24     if (promote_result != DataType::UNDEFINED) {
25         key_set.dtype = promote_result;
26     }
27 }
28
29 void operator()(const Tensor& x) {
30     const auto* tensor = x.impl().get();
31     if (tensor) {
32         AssignKernelKeySet(*tensor);
33     }
34 }
35
36 void operator()(const std::vector<Tensor>& x) {
37     const phi::TensorBase& tensor = *x.at(0).impl();
38     key_set.backend_set =
39         key_set.backend_set | detail::GetTensorBackendSet(tensor);
40     // TODO(chenweihang): select multi layout and dtype
41     key_set.layout = tensor.layout();
42     key_set.dtype = tensor.dtype();
43 }
44
45 void operator()(const paddle::optional<Tensor>& x) {
46     if (x) {
47         const phi::TensorBase& tensor = *(x.get_ptr()->impl());
48         AssignKernelKeySet(tensor);
49     }
50 }
51
52 // skip other type args, these args don't used in kernel selection
53 template <typename T>
54 void operator()(const T& x) {
55     // do nothing
56 }
57 };
```

2. 选择kernel

```
1 auto kernel_result =
  phi::KernelFactory::Instance().SelectKernelOrThrowError(
2     "matmul", {kernel_backend, kernel_layout, kernel_data_type});
3 const auto& kernel = kernel_result.kernel
4
5
6 KernelResult KernelFactory::SelectKernelOrThrowError(
7     const std::string& kernel_name,
8     const KernelKey& kernel_key,
9     bool use_gpubnn) const {
10 auto iter = kernels_.find(kernel_name);
11 PADDLE_ENFORCE_NE(
12     iter,
13     kernels_.end(),
14     phi::errors::NotFound("The kernel `%s` is not registered.",
15 kernel_name));
16
17 #if defined(PADDLE_WITH_CUDA) || defined(PADDLE_WITH_HIP)
18 if (use_gpubnn && kernel_key.backend() == Backend::GPU) {
19     auto kernel_iter = iter->second.find(
20         {Backend::GPUDNN, kernel_key.layout(), kernel_key.dtype()});
21     if (kernel_iter == iter->second.end() &&
22         kernel_key.layout() != phi::DataLayout::ALL_LAYOUT) {
23         kernel_iter = iter->second.find(
24             {Backend::GPUDNN, DataLayout::ALL_LAYOUT,
25 kernel_key.dtype()});
26     }
27     if (kernel_iter != iter->second.end()) {
28         return {kernel_iter->second, false};
29     }
30     LOG(WARNING) << "The cudnn kernel for [" << kernel_name
31         << "] is not registered.";
32 }
33 #endif
34
35 auto kernel_iter = iter->second.find(kernel_key);
36 // TODO(chenweihang): polish refine impl here
37 if (kernel_iter == iter->second.end() &&
38     kernel_key.layout() != phi::DataLayout::ALL_LAYOUT) {
39     phi::KernelKey any_layout_kernel_key(
40         kernel_key.backend(), phi::DataLayout::ALL_LAYOUT,
41         kernel_key.dtype());
```

```
39     kernel_iter = iter->second.find(any_layout_kernel_key);
40 }
41
42 PADDLE_ENFORCE_NE(
43     kernel_iter == iter->second.end() && kernel_key.backend() ==
Backend::CPU,
44     true,
45     phi::errors::NotFound(
46         "The kernel with key %s of kernel `%s` is not registered.",
47         kernel_key,
48         kernel_name));
49
50 if (FLAGS_enable_api_kernel_fallback && kernel_iter == iter-
>second.end()) {
51     // Fallback CPU backend
52     phi::KernelKey cpu_kernel_key(
53         phi::Backend::CPU, kernel_key.layout(), kernel_key.dtype());
54     kernel_iter = iter->second.find(cpu_kernel_key);
55     if (kernel_iter == iter->second.end() &&
56         kernel_key.layout() != phi::DataLayout::ALL_LAYOUT) {
57         phi::KernelKey any_layout_kernel_key(
58             phi::Backend::CPU, phi::DataLayout::ALL_LAYOUT,
kernel_key.dtype());
59         kernel_iter = iter->second.find(any_layout_kernel_key);
60     }
61
62     PADDLE_ENFORCE_NE(
63         kernel_iter,
64         iter->second.end(),
65         phi::errors::NotFound(
66             "The kernel with key %s of kernel `%s` is not registered
and"
67             " fail to fallback to CPU one.",
68             kernel_key,
69             kernel_name));
70
71     VLOG(3) << "missing " << kernel_key.backend() << " kernel: " <<
kernel_name
72             << ", expected_kernel_key:" << kernel_key
73             << ", fallbacking to CPU one!";
74
75     return {kernel_iter->second, true};
76 }
77
```

```
78 PADDLE_ENFORCE_NE(  
79     kernel_iter,  
80     iter->second.end(),  
81     phi::errors::NotFound(  
82         "The kernel with key %s of kernel `%s` is not registered and"  
83         " the current value of FLAGS_enable_api_kernel_fallback(bool,"  
84         " default true) is false. If you want to fallback this kernel"  
85         " to CPU one, please set the flag true before run again.",  
86         kernel_key,  
87         kernel_name));  
88  
89 return {kernel_iter->second, false};  
90 }
```

3. 获取执行硬件

</>

C++ | 收起 ^

```
1 auto* dev_ctx = GetDeviceContextByBackend(kernel_result.has_fallback_cpu  
? Backend::CPU : kernel_backend);  
2  
3  
4 phi::DeviceContext* GetDeviceContextByBackend(phi::Backend backend) {  
5     auto& pool = paddle::experimental::DeviceContextPool::Instance();  
6     return pool.GetMutable(phi::TransToPhiPlace(backend));  
7 }  
8  
9  
10 phi::Place TransToPhiPlace(const Backend& backend, bool set_device_id) {  
11     // NOTE(zhiqu): GetCurrentDeviceId not always success, and device id  
is not  
12     // always needed.  
13     // So, add set_device_id parameter here.  
14     switch (backend) {  
15         case phi::Backend::CPU:  
16             return phi::CPUPlace();  
17 #if defined(PADDLE_WITH_CUDA) || defined(PADDLE_WITH_HIP)  
18         case phi::Backend::GPU:  
19             return phi::GPUPlace(  
20                 set_device_id ? phi::backends::gpu::GetCurrentDeviceId() : 0);  
21 #endif  
22 #ifdef PADDLE_WITH_MKLDNN  
23         case phi::Backend::ONEDNN:  
24             return phi::CPUPlace();
```

```
25 #endif
26 #if defined(PADDLE_WITH_CUDA) || defined(PADDLE_WITH_HIP)
27     case phi::Backend::GPUDNN:
28         return phi::GPUPlace(
29             set_device_id ? phi::backends::gpu::GetCurrentDeviceId() : 0);
30 #endif
31 #if defined(PADDLE_WITH_XPU)
32     case phi::Backend::XPU:
33         return phi::XPUPlace(
34             set_device_id ? phi::backends::xpu::GetXPUCurrentDeviceId() :
35             0);
36 #endif
37     case phi::Backend::KPS:
38 #if defined(PADDLE_WITH_CUDA) || defined(PADDLE_WITH_HIP)
39     return phi::GPUPlace(
40         set_device_id ? phi::backends::gpu::GetCurrentDeviceId() : 0);
41 #elif defined(PADDLE_WITH_XPU_KP)
42     return phi::XPUPlace(
43         set_device_id ? phi::backends::xpu::GetXPUCurrentDeviceId() :
44         0);
45 #endif
46     default: {
47 #ifdef PADDLE_WITH_CUSTOM_DEVICE
48         size_t device_type_id_ = static_cast<size_t>(backend) -
49             static_cast<size_t>
50             (Backend::NUM_BACKENDS);
51         std::string device_type =
52             phi::GetGlobalDeviceType(device_type_id_);
53         if (!device_type.empty()) {
54             return phi::CustomPlace(
55                 device_type,
56                 set_device_id ? phi::DeviceManager::GetDevice(device_type) :
57                 0);
58         }
59 #endif
60     PADDLE_THROW(phi::errors::Unimplemented(
61         "Unsupported backend `%s` when casting it to paddle place
62         type.",
63         backend));
64     }
```

4. 获取输入tensor数据

</>

C++ | 收起 ^

```
1 auto input_x = PrepareData(x, kernel.InputAt(0), {});
2
3
4 std::shared_ptr<phi::DenseTensor> PrepareData(
5     const Tensor& input,
6     const phi::TensorArgDef& target_args_def,
7     const TransformFlag& transform_flag) {
8     const auto& tensor_in = input.impl();
9     if (tensor_in) {
10         phi::DenseTensor& dense_tensor =
11             *static_cast<phi::DenseTensor*>(tensor_in.get());
12         if (!transform_flag.NeedTransform() || !dense_tensor.initialized()
13             ||
14             (!NeedTransformPlace(
15                 dense_tensor.place(), target_args_def.backend,
16                 transform_flag) &&
17                 !NeedTransformDataType(
18                     dense_tensor.dtype(), target_args_def.dtype,
19                     transform_flag) &&
20                 !NeedTransformLayout(dense_tensor.layout(),
21                                     target_args_def.layout,
22                                     dense_tensor.place(),
23                                     transform_flag)))) {
24             return std::static_pointer_cast<phi::DenseTensor>(tensor_in);
25         }
26         phi::DenseTensor out =
27             TransformData(&dense_tensor, target_args_def, transform_flag);
28         return std::make_shared<phi::DenseTensor>(std::move(out));
29     }
30     return nullptr;
31 }
32
33 phi::DenseTensor TransformData(phi::DenseTensor* tensor,
34     const phi::TensorArgDef& target_args_def,
35     const TransformFlag& transform_flag) {
36     phi::DenseTensor out = *tensor;
37     bool trans_layout = false;
38     bool trans_dtype = false;
39     if (NeedTransformLayout(tensor->layout(),
40                             target_args_def.layout,
```



```

40         tensor->place(),
41         transform_flag)) {
42     out = TransDataLayout(out, target_args_def.layout);
43     trans_layout = true;
44 }
45
46 if (NeedTransformDataType(
47     tensor->dtype(), target_args_def.dtype, transform_flag)) {
48     out = TransDataType(out, target_args_def.dtype);
49     trans_dtype = true;
50 }
51
52 if (NeedTransformPlace(
53     out.place(), target_args_def.backend, transform_flag)) {
54     out = TransDataPlace(out,
55 phi::TransToPhiPlace(target_args_def.backend));
56     if (!trans_layout && !trans_dtype &&
57         tensor->place().GetType() == AllocationType::GPUPINNED) {
58         tensor->ShareBufferWith(out);
59     }
60 }
61 return out;
62 }

```

5. RecordOpInfoSupplement

</>

C++ | 收起 ^

```

1 RecordOpInfoSupplement::RecordOpInfoSupplement(
2     const std::string &type,
3     const std::vector<std::pair<const char *,
4     std::vector<framework::DDim>>>
5     &input_shapes) {
6     if (FLAGS_enable_host_event_recorder_hook == false) {
7         return;
8     }
9     if (IsEnabled() == false) {
10        return;
11    }
12    std::map<std::string, std::vector<framework::proto::VarType::Type>>
13    dtypes;
14    std::vector<std::string> callstack;

```

```

13 HostEventRecorder<OperatorSupplementOriginEvent>::GetInstance().RecordEvent(
14     PosixInNsec(), type, input_shapes, dtypes, callstack);
15 }

```

6. 设置输出tensor的数据指针

C++ | 收起 ^

```

1 phi::DenseTensor* SetKernelOutput(Tensor* out) {
2     if (out) {
3         if (out->impl() == nullptr) {
4             out->set_impl(std::make_shared<phi::DenseTensor>());
5         }
6         return static_cast<phi::DenseTensor*>(out->impl().get());
7     }
8     return nullptr;
9 }

```

7. 调用InferMeta 函数 （实现算子时需实现InferMeta函数）

C++ | 收起 ^

```

1 void MatmulInferMeta(const MetaTensor& x,
2                     const MetaTensor& y,
3                     bool trans_x,
4                     bool trans_y,
5                     MetaTensor* out) {
6     std::vector<int64_t> dims_x = phi::vectorize(x.dims());
7     std::vector<int64_t> dims_y = phi::vectorize(y.dims());
8     auto ndims_x = dims_x.size();
9     auto ndims_y = dims_y.size();
10    PADDLE_ENFORCE_GT(ndims_x,
11                      0UL,
12                      phi::errors::InvalidArgument(
13                          "The Input(x) dims size must be greater than 0,"
14                          " but reviced dims size is 0. "));
15    PADDLE_ENFORCE_GT(ndims_y,
16                      0UL,
17                      phi::errors::InvalidArgument(
18                          "The Input(y) dims size must be greater than 0,"
19                          " but reviced dims size is 0. "));
20
21    bool x_broadcasted = false, y_broadcasted = false;

```

```
22  if (ndims_x == 1) {
23      dims_x.insert(dims_x.begin(), 1);
24      ndims_x = 2;
25      x_broadcasted = true;
26  }
27
28  if (ndims_y == 1) {
29      dims_y.push_back(1);
30      ndims_y = 2;
31      y_broadcasted = true;
32  }
33
34  size_t M, N;
35  if (trans_x) {
36      M = dims_x[ndims_x - 1];
37  } else {
38      M = dims_x[ndims_x - 2];
39  }
40  if (trans_y) {
41      N = dims_y[ndims_y - 2];
42  } else {
43      N = dims_y[ndims_y - 1];
44  }
45
46  std::vector<int64_t> new_dims;
47  if (ndims_x > ndims_y) {
48      new_dims.assign(dims_x.begin(), dims_x.end() - 2);
49  } else if (ndims_x < ndims_y) {
50      new_dims.assign(dims_y.begin(), dims_y.end() - 2);
51  } else {
52      new_dims.reserve(ndims_x);
53      for (size_t i = 0; i < ndims_x - 2; ++i) {
54          new_dims.push_back(std::max(dims_x[i], dims_y[i]));
55      }
56  }
57  if (!x_broadcasted) {
58      new_dims.push_back(M);
59  }
60  if (!y_broadcasted) {
61      new_dims.push_back(N);
62  }
63  if (x_broadcasted && y_broadcasted) {
64      new_dims.push_back(1);
```

```

65     }
66
67     auto ddim_out = phi::make_ddim(new_dims);
68
69     out->set_dims(ddim_out);
70     out->set_dtype(x.dtype());
71     out->set_layout(x.layout());
72 }

```

8. 获得kernel的声明格式 -> 调用运算kernel函数

```

</> C++ | 收起 ^
1 using kernel_signature = void (*)(const platform::DeviceContext&, const
  phi::DenseTensor&, const phi::DenseTensor&, bool, bool,
  phi::DenseTensor*);
2 auto* kernel_fn = kernel.GetVariadicKernelFn<kernel_signature>();
3
4 template <typename Fn>
5 Fn GetVariadicKernelFn() const {
6 auto* func = reinterpret_cast<Fn>(variadic_fn_);
7 return func;
8 }

```

9. 如果需要返回cpu端，将输出转换为cpu数据

```

</> C++ | 收起 ^
1 void TransDataBackend(const phi::DenseTensor* tensor,
2                       Backend target_backend,
3                       phi::DenseTensor* out) {
4     if (tensor && tensor->initialized()) {
5         *out = TransDataPlace(*tensor,
6 phi::TransToPhiPlace(target_backend));
7     }
8
9 //TransToPhiPlace 在获取硬件阶段使用过
10 inline phi::DenseTensor TransDataPlace(const phi::DenseTensor& tensor,
11                                         Place dst_place) {
12     VLOG(3) << "DeviceTransform in, src_place " << tensor.place()
13         << " dst_place: " << dst_place;
14
15     DefaultAllocator alloc(dst_place);
16     phi::DenseTensor out(&alloc,

```

```

17         {tensor.dtype(), tensor.dims(),
    tensor.layout()});
18
19 #if defined(PADDLE_WITH_CUDA) || defined(PADDLE_WITH_HIP)
20     auto& pool = paddle::platform::DeviceContextPool::Instance();
21     // NOTE(yy): TransDataPlace should wait for computation of input.
22     if (!platform::is_cuda_pinned_place(tensor.place())) {
23         pool.Get(tensor.place())->Wait();
24         pool.Get(dst_place)->Wait();
25     }
26 #endif
27
28     // FIXME(zcd): TransDataPlace is used to transform data from GPU to
    CPU and
29     // the enforced checkings have been done in GetDeviceContext, so the
30     // `dev_ctx->Wait()` is necessary. But `dev_ctx->Wait()` will make the
    program
31     // slow, especially when the number of elements is little, for
    example,
32     // the elements of learning rate are one and it's CPU side.
33     // One solution is to use a CUDA kernel to complete the copy operation
    when
34     // the transforming is from CPU to GPU and the number of elements is
    little.
35     // But the embarrassment is that this solution this solution makes
    training
36     // slower.
37     paddle::framework::TensorCopySync(tensor, dst_place, &out);
38     return out;
39 }

```

5 matmul反向节点代码

反向算子以类进行管理，类中的operator函数为实际的反向算子调用入口，因此头文件中存放的是类的声明。

1.在 `paddle/fluid/eager/api/generated/eager_generated/nodes/nodes.h` 中，实现了函数的反向算子类

每个反向算子的类继承于GradNodeBase类；

私有成员包含：

- 是反向输入tensor的前向输入tensor（类型为TensorWrapper）
- 用户传参map
- 默认传参map

公有成员包含:

- 继承自基类的默认构造函数，有输入输出tensor个数的构造函数及析构函数,各函数中分别加入调试VLOG信息" Construct xxx" ," Destruct xxx"
- 重写基类的operator()函数（注意此处需要将参数 使用"/NOLINT注释掉以屏蔽pre-commit的代码规范检查）
- 重写基类的ClearTensorWrappers()函数对TensorWrapper的私有成员依次清理后，设置已清理标志
- 重写基类的name()函数获取此节点的实际名称
- 重写拷贝构造函数Copy(), 使用shared_ptr引用新建反向节点实例
- 分别实现设置输入tensor为TensorWrapper的函数：
SetTensorWrapperX;SetTensorWrapperY;
- 分别实现设置参数函数：SetAttributetranspose_x; SetAttributetranspose_y

</>

C++ | 收起 ^

```
1 class MatmulGradNodeFinal : public egr::GradNodeBase {
2 public:
3     MatmulGradNodeFinal() : egr::GradNodeBase() {}
4     MatmulGradNodeFinal(size_t bwd_in_slot_num, size_t bwd_out_slot_num) :
5         egr::GradNodeBase(bwd_in_slot_num, bwd_out_slot_num) {}
6     ~MatmulGradNodeFinal() override = default;
7
8     virtual
9         paddle::small_vector<std::vector<paddle::experimental::Tensor>,
10         egr::kSlotSmallVectorSize> operator()(
11             paddle::small_vector<std::vector<paddle::experimental::Tensor>,
12             egr::kSlotSmallVectorSize>& grads, bool create_graph = false, bool
13             is_new_grad = false) override;
14     std::string name() override { return "MatmulGradNodeFinal"; }
15
16     void ClearTensorWrappers() override {
17         x_.clear();
18         y_.clear();
19
20         SetIsTensorWrappersCleared(true);
21     }
22
23     std::shared_ptr<GradNodeBase> Copy() const override {
24         auto copied_node = std::shared_ptr<MatmulGradNodeFinal>(new
25             MatmulGradNodeFinal(*this));
26         return copied_node;
27     }
28 }
```

```

24 // SetTensorWrapperX, SetTensorWrapperY, ...
25 void SetTensorWrapperx(const paddle::experimental::Tensor& x) {
26     x_ = egr::TensorWrapper(x, false);
27 }
28 void SetTensorWrappery(const paddle::experimental::Tensor& y) {
29     y_ = egr::TensorWrapper(y, false);
30 }
31
32 // SetAttributes
33 void SetAttributetranspose_x(const bool& transpose_x) {
34     transpose_x_ = transpose_x;
35 }
36 void SetAttributetranspose_y(const bool& transpose_y) {
37     transpose_y_ = transpose_y;
38 }
39
40 private:
41 // TensorWrappers
42 egr::TensorWrapper x_;
43 egr::TensorWrapper y_;
44
45 // Attributes
46 bool transpose_x_ = false;
47 bool transpose_y_ = false;
48 };

```

在 `paddle/fluid/eager/api/generated/` `eager_generated/nodes/nodes.h` 中实现后向类的operator算子

设置gradient hook -> 设置输入Tensor及参数 -> 设置输出Tensor -> 运行时检查是否需要高阶反向 -> 调用反向c++ API,检查输出的合法性 -> 设置输出tensor的AutoGradMeta -> 创建二阶反向节点

</>

C++ | 收起 ^

```

1 paddle::small_vector<std::vector<paddle::experimental::Tensor>,
  egr::kSlotSmallVectorSize> MatmulGradNodeFinal::operator()
2 (paddle::small_vector<std::vector<paddle::experimental::Tensor>,
  egr::kSlotSmallVectorSize>& grads, bool create_graph, bool is_new_grad)
  {
3 // Fill Zero For GradIn Tensors
4
5 // 设置gradient hook

```



```
6 auto hooked_grads = ApplyGradientHooks(grads);
7
8 // 设置输入Tensor及参数
9 auto x = egr::EagerUtils::RecoverTensorWrapper(&this->x_);
10 auto y = egr::EagerUtils::RecoverTensorWrapper(&this->y_);
11 auto& grad_out = hooked_grads[0][0];
12 auto& transpose_x = this->transpose_x_;
13 auto& transpose_y = this->transpose_y_;
14
15 // 设置输出Tensor
16 const auto& out metas = OutputMeta();
17 paddle::small_vector<std::vector<paddle::experimental::Tensor>,
egr::kSlotSmallVectorSize> returns(2);
18 for (int i = 0; i < 2; ++i) {
19     out_metas[i].size() == 0 ? returns[i].resize(1) :
returns[i].resize(out_metas[i].size());
20 }
21
22 auto* api_output_0 = (out_metas[0].empty() || out_metas[0]
[0].IsStopGradient()) ? nullptr : &returns[0][0];
23 auto* api_output_1 = (out_metas[1].empty() || out_metas[1]
[0].IsStopGradient()) ? nullptr : &returns[1][0];
24 // 运行时检查是否需要下一个梯度
25 bool trace_backward = egr::Controller::Instance().HasGrad() &&
create_graph;
26
27 // Inplace Check
28
29 // Inplace Strategy
30
31 // 调用反向c++ API
32 VLOG(3) << "Final State Running: MatmulGradNodeFinal";
33
34 paddle::experimental::matmul_grad(x, y, grad_out, transpose_x,
transpose_y, api_output_0, api_output_1);
35 // 检查输出的合法性
36 if (FLAGS_check_nan_inf) { egr::CheckTensorHasNanOrInf("matmul_grad",
returns); }
37
38 // 设置输出tensor的AutoGradMeta
39 auto& grad_x = returns[0][0];
40 egr::AutogradMeta* grad_x_autograd_meta = returns[0][0].initialized()
? egr::EagerUtils::autograd_meta(&grad_x) : nullptr;
```

```
41  if (grad_x_autograd_meta) grad_x_autograd_meta-  
    >SetStopGradient(false);  
42  
43  auto& grad_y = returns[1][0];  
44  egr::AutogradMeta* grad_y_autograd_meta = returns[1][0].initialized()  
    ? egr::EagerUtils::autograd_meta(&grad_y) : nullptr;  
45  if (grad_y_autograd_meta) grad_y_autograd_meta-  
    >SetStopGradient(false);  
46  
47  // 创建二阶反向节点  
48  if(trace_backward) {  
49      paddle::platform::RecordEvent  
        node_creation_record_event("matmul_grad node_creation",  
        paddle::platform::TracerEventType::OperatorInner, 1);  
50  
51      // Node Construction  
52      auto grad_node = std::shared_ptr<MatmulDoubleGradNodeFinal>(new  
        MatmulDoubleGradNodeFinal(2, 3));  
53      // SetAttributes if needed  
54      grad_node->SetAttributetranspose_x(transpose_x);  
55      grad_node->SetAttributetranspose_y(transpose_y);  
56      // Set TensorWrappers for Forward Inputs if needed  
57      grad_node->SetTensorWrapperx(x);  
58      grad_node->SetTensorWrappery(y);  
59      grad_node->SetTensorWrappergrad_out(grad_out);  
60      // SetGradOutMeta & SetEdges  
61      grad_node->SetGradOutMeta(x, 0);  
62      grad_node->SetGradOutMeta(y, 1);  
63      grad_node->SetGradOutMeta(grad_out, 2);  
64      // SetOutRank & SetHistory & SetGradInMeta & RetainGrad  
65      if (grad_x_autograd_meta) {  
66          egr::EagerUtils::SetOutRankWithSlot(grad_x_autograd_meta, 0);  
67      }  
68      if (grad_y_autograd_meta) {  
69          egr::EagerUtils::SetOutRankWithSlot(grad_y_autograd_meta, 1);  
70      }  
71      if (grad_x_autograd_meta) {  
72          egr::EagerUtils::SetHistory(grad_x_autograd_meta, grad_node);  
73      }  
74      if (grad_y_autograd_meta) {  
75          egr::EagerUtils::SetHistory(grad_y_autograd_meta, grad_node);  
76      }  
77      grad_node->SetGradInMeta(grad_x, 0);  
78      grad_node->SetGradInMeta(grad_y, 1);
```

```
79     egr::EagerUtils::CheckAndRetainGrad(grad_x);
80     egr::EagerUtils::CheckAndRetainGrad(grad_y);
81     // Set TensorWrappers for Forward Outputs if needed
82
83 }
84
85 // Return
86 if(NeedComplexToRealConversion())
    HandleComplexGradToRealGrad(&returns);
87 return returns;
88
89 }
```

1. 设置gradient hook

</>

C++ | 收起 ^

```
1 auto hooked_grads = ApplyGradientHooks(grads);
2
3
4 paddle::small_vector<std::vector<paddle::experimental::Tensor>,
5                       kSlotSmallVectorSize>
6 GradNodeBase::ApplyGradientHooks(
7     const
8     paddle::small_vector<std::vector<paddle::experimental::Tensor>,
9                             kSlotSmallVectorSize>& tensors) {
9     //按照参数Tensor的大小创建一个输出vector
10    paddle::small_vector<std::vector<paddle::experimental::Tensor>,
11                          kSlotSmallVectorSize>
12        outs(tensors.size());
13    //遍历Grad_node类的gradient_hooks_,对outs设置hook后,将tensor的值赋给outs输出
14    for (auto& hook_pair : gradient_hooks_) {
15        size_t slot_id = std::get<0>(hook_pair.second);
16        size_t rank = std::get<1>(hook_pair.second);
17        auto hook = std::get<2>(hook_pair.second);
18
19        PADDLE_ENFORCE(slot_id < tensors.size(),
20                        paddle::platform::errors::Fatal(
21                            "Slot_id from registered hook should be smaller
22                            than "
23                                "slot size of grad_tensors"));
24        PADDLE_ENFORCE(rank < tensors[slot_id].size(),
25                        paddle::platform::errors::Fatal(
```

```

26         "rank of slot %d from registered hook should be
    smaller "
27         "than rank size of grad_tensors",
28         slot_id));
29
30     std::vector<paddle::experimental::Tensor>& slot_out = outs[slot_id];
31     slot_out.resize(tensors[slot_id].size());
32     paddle::experimental::Tensor& out = slot_out[rank];
33     if (!out.defined() || !out.initialized()) {
34         out = (*hook)(tensors[slot_id][rank]);
35     } else {
36         // If more than one hook is registered, the input to the next hook
    func
37         // should be the output of the previous hook
38         out = (*hook)(out);
39     }
40 }
41
42 for (size_t i = 0; i < outs.size(); i++) {
43     if (outs[i].empty() && (!tensors[i].empty())) {
44         outs[i].resize(tensors[i].size());
45     }
46     // TODO(Jiabin): Optimize this if we only add hook slot by slot
47     for (size_t j = 0; j < outs[i].size(); j++) {
48         if (!outs[i][j].defined() || !outs[i][j].initialized()) {
49             outs[i][j] = tensors[i][j];
50         }
51         CheckTensor(tensors[i][j], outs[i][j]);
52     }
53 }
54 return outs;
55 }

```

2. 设置输入Tensor及参数

</>

C++ | 收起 ^

```

1 auto x = egr::EagerUtils::RecoverTensorWrapper(&this->x_);
2 auto y = egr::EagerUtils::RecoverTensorWrapper(&this->y_);
3 auto& grad_out = hooked_grads[0][0];
4 auto& transpose_x = this->transpose_x_;
5 auto& transpose_y = this->transpose_y_;
6
7

```

```
8 paddle::experimental::Tensor EagerUtils::RecoverTensorWrapper(  
9     TensorWrapper* tw) {  
10     return tw->recover();  
11 }  
12  
13 paddle::experimental::Tensor recover() {  
14     VLOG(6) << "Recover tensor: " << intermediate_tensor_.name()  
15         << " for wrapper";  
16     if (!intermediate_tensor_.defined()) {  
17         VLOG(6) << "Return NULL tensor Here. ";  
18         return paddle::experimental::Tensor();  
19     }  
20 #ifndef PADDLE_NO_PYTHON  
21     if (packed_value_ && unpack_hook_) {  
22         auto tensor_unpacked =  
23             (*unpack_hook_)(reinterpret_cast<void*>(packed_value_));  
24         auto src_dense_tensor =  
25             static_cast<phi::DenseTensor*>(tensor_unpacked.impl().get());  
26         static_cast<phi::DenseTensor*>(intermediate_tensor_.impl().get())  
27             ->ResetHolder(src_dense_tensor->MoveMemoryHolder());  
28     } else {  
29 #endif  
30         check_inplace_version();  
31 #ifndef PADDLE_NO_PYTHON  
32     }  
33 #endif  
34  
35     paddle::experimental::Tensor recovered_tensor =  
36         intermediate_tensor_;  
37     std::shared_ptr<GradNodeBase> new_grad_node =  
38         weak_grad_node_.lock();  
39     if (new_grad_node) {  
40         VLOG(7) << "Recovered TensorWrapper with GradNode "  
41             << new_grad_node->name() << " addr: " <<  
42             new_grad_node.get();  
43     } else {  
44         VLOG(7) << "Recovered TensorWrapper with Empty GradNode";  
45     }  
46     auto* intermediate_autograd_meta =  
47         EagerUtils::nullable_autograd_meta(intermediate_tensor_);  
48     if (intermediate_autograd_meta) {
```

```
48     auto p_ab_autograd_meta =
49         std::make_shared<AutogradMeta>(*intermediate_autograd_meta);
50     if (new_grad_node) {
51         p_ab_autograd_meta->SetGradNode(new_grad_node);
52     }
53     recovered_tensor.set_autograd_meta(p_ab_autograd_meta);
54 }
55 return recovered_tensor;
56 }
```

3. 设置输出Tensor

</>

C++ | 收起 ^

```
1 const auto& out_metas = OutputMeta(); //取出反向节点的bwd_out_meta
2 paddle::small_vector<std::vector<paddle::experimental::Tensor>,
  egr::kSlotSmallVectorSize> returns(2); //定义两个输出
3 for (int i = 0; i < 2; ++i) { //根据bwd_out_meta设置输出tensor的大小
4     out_metas[i].size() == 0 ? returns[i].resize(1) :
      returns[i].resize(out_metas[i].size());
5 }
6
7 auto* api_output_0 = (out_metas[0].empty() ||
  out_metas[0][0].IsStopGradient()) ? nullptr : &returns[0][0];
8 auto* api_output_1 = (out_metas[1].empty() ||
  out_metas[1][0].IsStopGradient()) ? nullptr : &returns[1][0];
```

4. 运行时检查是否需要高阶反向

</>

C++ | 收起 ^

```
1 bool trace_backward = egr::Controller::Instance().HasGrad() &&
  create_graph;
```

5. 调用反向c++ API,检查输出的合法性

</>

C++ | 收起 ^

```
1 paddle::experimental::matmul_grad(x, y, grad_out, transpose_x,
  transpose_y, api_output_0, api_output_1);
2 if (FLAGS_check_nan_inf) { egr::CheckTensorHasNanOrInf("matmul_grad",
  returns); }
```

6. 设置输出tensor的AutoGradMeta

</>

C++ | 收起 ^

```
1 auto& grad_x = returns[0][0]; //取出计算得到的Tensor
```

```
2 egr::AutoGradMeta* grad_x_autograd_meta = returns[0][0].initialized() ?  
  egr::EagerUtils::autograd_meta(&grad_x) : nullptr; //设置输出tensor的  
  AutoGradMeta  
3 if (grad_x_autograd_meta) grad_x_autograd_meta->SetStopGradient(false);
```

7. 创建二阶反向节点

</>

C++ | 收起 ^

```
1 if(trace_backward) {  
2   略  
3 }
```